Pimpri Chinchwad Education Trust's
Pimpri Chinchwad College of Engineering
Department of Information Technology

Agile Software Development Laboratory

2026-2027

FA2 — SKILL-BASED ACTIVITY

Agile Project Plan — Yum Food (Online Food Ordering System)

STUDENT NAME Tony Prajapati	**ROLL NO. / PRN** 123B1F079
BATCH / DIVISION Division B2	**SEMESTER** Semester VII
COURSE CODE BIT27PE11	**SUBJECT** Agile Software Development Laboratory
FACULTY / GUIDE Mrs. Radha Deoghare	**SUBMISSION DATE** 17 October 2026

Activity: FA2 — Skill-Based Activity: Design and Demonstrate a Simple CI/CD Pipeline for an Agile Project

Group members: Tony Prajapati (123B1F079) · Pratik Rajput (123B1F135)

Project repository: github.com/gittonyp/YumFood-CICD

Technology stack: MongoDB · Express · React · Node.js (MERN) · Jest (testing) · ESLint (code quality) · GitHub Actions (CI/CD)

Sprint length: 1 week — a short setup sprint (Sprint 0) followed by two development sprints

1. Project Overview

Yum Food is an online food ordering web application. Customers register and sign in, browse the food menu, add items to a shopping cart, place orders and track the status of those orders. An administrator maintains the menu (add / update / delete items) and moves each order through its lifecycle (`pending → accepted / declined`).

The team works in one-week sprints with a prioritized product backlog. Every user story is "done" only when its acceptance criteria are met, tests pass and the CI/CD pipeline is green — the same pipeline that is designed and demonstrated for this FA2 activity.

2. Product Backlog — User Stories

ID	User story	Priority	Story points
US-01	As a **new customer**, I want to **register an account and log in**, so that **I can place orders that belong to me**.	Must	3
US-02	As a **customer**, I want to **browse the food menu with names, images and prices**, so that **I can decide what to order**.	Must	2
US-03	As a **customer**, I want to **add items to my cart and place an order**, so that **my food can be prepared and delivered to me**.	Must	8
US-04	As a **customer**, I want to **view my order history and current order status**, so that **I know whether my order is pending, accepted or declined**.	Should	3
US-05	As an **administrator**, I want to **add, update and delete menu items**, so that **customers always see the current menu**.	Must	5
EN-01	As the **development team**, I want **automated build, testing, code-quality checks and deployment on every push / pull request**, so that **defects are caught early and every release is reproducible**.	Must (enabler)	5

Total: 26 story points — 21 across the 5 user stories plus 5 for the technical enabler.

---

3. Sprint Planning

Team capacity: 2 members × ~6 focused hours per week ≈ 12 hours ⇒ a conservative plan of **≤ 13 story points per sprint**.

Sprint 0 — Setup (1–2 days)

Sprint goal: A working repository with the first green pipeline.

Task	Owner
Set up the project repository, enable GitHub Actions, protect the `main` branch	Both
Add CI workflow skeleton: checkout → install dependencies → build	Member A
Set up the task board (GitHub Projects: To Do / In Progress / In Review / Done)	Member B

Deliverable: Pipeline triggers on every push and pull request and completes successfully.

Sprint 1 — Accounts & Menu (Week 1)

Sprint goal: A visitor can register, log in and browse the menu; an admin can manage menu items; the CI pipeline runs build, tests and lint on every pull request.

Backlog item	Points	Owner
US-01 — registration & login	3	Member A
US-02 — browse menu	2	Member B
US-05 — admin menu management	5	Member A
EN-01 (part 1) — build, test and lint stages in the pipeline	3	Member B

Planned: 13 points. **Sprint review:** demo registration, login and menu management; **retrospective:** keep / improve actions recorded for Sprint 2.

Sprint 2 — Ordering & Tracking (Week 2)

Sprint goal: A logged-in customer can order end-to-end and track the status of the order; the pipeline deploys the application automatically after all quality gates pass.

Backlog item	Points	Owner
US-03 — cart & place order	8	Member B
US-04 — order history & status tracking	3	Member A
EN-01 (part 2) — deployment stage + branch protection demonstration	2	Both

Planned: 13 points. **Sprint review:** end-to-end order placement and status update demo; **retrospective** and final submission checklist.

Ceremonies & working agreements

- **Sprint planning** at the start and **sprint review + retrospective** at the end of each week.
- **Daily stand-up** (15 minutes): what was done, what is next, any blockers.
- **Branching:** work happens on `feature/US-xx-short-name` branches; pull requests require **1 review by the other member** and a **green pipeline** before merging into `main`.

---

4. Definition of Done (DoD)

A backlog item is **Done** only when **all** of the following are true:

Code & review

- [ ] Code is committed on a `feature/...` branch and pushed to GitHub.
- [ ] A pull request is open, describes the change and is reviewed and approved by the other member.
- [ ] No unresolved review comments; no directly pushed commits to `main` (branch protection).

Testing & quality

- [ ] Unit / integration tests are added for the new behaviour (Jest + Supertest).
- [ ] The complete test suite passes via `npm test` (backend, integration and frontend projects).
- [ ] ESLint reports **zero errors**.
- [ ] The GitHub Actions pipeline (build → test → lint) is **green** on the pull request.

Functionality & delivery

- [ ] All acceptance criteria of the user story are demonstrated and verified.
- [ ] The feature is merged into `main` and the **deployment stage of the pipeline succeeds**.
- [ ] No critical or high-severity bugs are known for the increment.
- [ ] Relevant documentation (README / this plan) is updated.

---

5. Acceptance Criteria

Written in **Given / When / Then** (Gherkin) format, as required for at least one user story — here for **US-01** and **US-03**.

US-01 — Registration & login

AC-1.1 Successful registration

- **Given** I am a new visitor on the sign-up page
- **When** I submit a name, a unique e-mail address and a valid password
- **Then** my account is created, I am logged in and I am taken to the menu page

AC-1.2 Duplicate e-mail rejected

- **Given** an account already exists with the e-mail `tony@example.com`
- **When** I try to register with the same e-mail
- **Then** registration is rejected with the message "Email already in use" and no account is created

AC-1.3 Login with valid credentials

- **Given** I have a registered account
- **When** I log in with the correct e-mail and password
- **Then** I am authenticated and my session token is returned

AC-1.4 Login with wrong password

- **Given** I have a registered account
- **When** I log in with an incorrect password
- **Then** the login is rejected with status 401 and no token is issued

US-03 — Cart & place order

AC-3.1 Add an item to the cart

- **Given** I am logged in and viewing the menu
- **When** I click "Add to cart" on "Margherita Pizza"
- **Then** the cart shows 1 × Margherita Pizza and the cart total equals its price

AC-3.2 Place an order

- **Given** my cart contains at least one item
- **When** I confirm the order
- **Then** an order is created with status `pending`, the item and price details are stored, and the cart is emptied

AC-3.3 Empty cart cannot be ordered

- **Given** my cart is empty
- **When** I try to confirm an order
- **Then** the system blocks it and shows "Your cart is empty"

AC-3.4 Order status is tracked

- **Given** I placed an order
- **When** I open "My Orders"
- **Then** I see the order and its current status (`pending`, `accepted` or `declined`)
- **And** when the administrator accepts the order, my order view shows the new status

---

6. CI/CD Workflow Alignment

The FA2 CI/CD workflow is itself tracked as enabler story **EN-01** and every stage acts as a quality gate for the Definition of Done:

Stage	Tooling	Gate behaviour
GitHub (source)	Feature branches, pull requests, protected `main`	Pipeline triggers on every push and PR
Build	`actions/checkout`, `actions/setup-node`, `npm ci`, frontend build	Fails if dependencies or build break
Automated Testing	Jest + Supertest with `mongodb-memory-server` (no external DB)	Fails if any unit / integration test fails
Code Quality Check	ESLint	Fails on lint errors
Deployment	Automated publish of the built application only after all previous stages pass on `main`	Artifact / service is updated automatically

Student Signature & Date

Faculty Signature & Date